

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет программной инженерии и компьютерной техники

Направление подготовки: 09.03.04 – Программная инженерия
Дисциплина «Вычислительная математика»

Лабораторная работа №5 «Интерполяция функции»

Вариант 17

Выполнил

Храбров Артём Алексеевич

Проверил

Наумова Надежда Александровна

Санкт-Петербург, 2026

Цель

Решить задачу интерполяции: найти значения функции при заданных значениях аргумента, отличных от узловых точек, используя интерполяционные многочлены Лагранжа, Ньютона и Гаусса.

1. Вычислительная реализация задачи

1.1 Исходные данные

Узловые точки:

i	0	1	2	3	4	5	6
x_i	0.50	0.55	0.60	0.65	0.70	0.75	0.80
y_i	1.5320	2.5356	3.5406	4.5462	5.5504	6.5559	7.5594

Узлы равноотстоящие с шагом $h = 0.05$. Необходимо найти значения функции в точках: $X_1 = 0.761$ и $X_2 = 0.661$.

1.2 Таблица конечных разностей

x_i	y_i	Δy	$\Delta^2 y$	$\Delta^3 y$	$\Delta^4 y$	$\Delta^5 y$	$\Delta^6 y$
0.50	1.5320	1.0036					
0.55	2.5356	1.0050	0.0014				
0.60	3.5406	1.0056	0.0006	-0.0008			
0.65	4.5462	1.0042	-0.0014	-0.0020	-0.0012		
0.70	5.5504	1.0055	0.0013	0.0027	0.0047	0.0059	
0.75	6.5559	1.0035	-0.0020	-0.0033	-0.0060	-0.0107	-0.0166
0.80	7.5594						

1.3 Интерполяция многочленом Ньютона для $X_1 = 0.761$

Точка $X_1 = 0.761$ находится в конце таблицы, поэтому нужно использовать формулу Ньютона для интерполяции назад.

$$P_n(x) = y_n + t\Delta y_{n-1} + \frac{t(t+1)}{2!}\Delta^2 y_{n-2} + \dots$$

$$t = \frac{0.761 - 0.80}{0.05} = -0.78$$

$$\begin{aligned}
y_6 &= 7.5594 \\
\Delta y_5 &= 1.0035 \\
\Delta^2 y_4 &= -0.0020 \\
\Delta^3 y_3 &= -0.0033
\end{aligned}$$

$$\begin{aligned}
P_3(0.761) &= y_6 + t\Delta y_5 + \frac{t(t+1)}{2}\Delta^2 y_4 + \frac{t(t+1)(t+2)}{6}\Delta^3 y_3 \\
&= 7.5594 - 0.78273 + 0.0001716 + 0.0001151 = 6.7769567 \approx \mathbf{6.7770}
\end{aligned}$$

1.4 Интерполяция многочленом Гаусса для $X_2 = 0.661$

Точка $X_2 = 0.661$ находится в середине таблицы. Выберем в качестве начального узел $x_0 = 0.65$ (индекс $i = 3$). Так как $X_2 > x_0$, применим формулу Гаусса для интерполирования вперёд.

$$P_n(x) = y_0 + t\Delta y_0 + \frac{t(t-1)}{2!}\Delta^2 y_{-1} + \frac{(t+1)t(t-1)}{3!}\Delta^3 y_{-1} + \dots$$

где $t = \frac{x-x_0}{h}$.

$$t = \frac{0.661 - 0.65}{0.05} = 0.22$$

Для вычислений используем центральные разности:

$$\begin{aligned}
y_0 &\rightarrow y_3 = 4.5462 \\
\Delta y_0 &\rightarrow \Delta y_3 = 1.0042 \\
\Delta^2 y_{-1} &\rightarrow \Delta^2 y_2 = -0.0014 \\
\Delta^3 y_{-1} &\rightarrow \Delta^3 y_2 = -0.0020
\end{aligned}$$

$$\begin{aligned}
P_3(0.661) &= y_3 + t\Delta y_3 + \frac{t(t-1)}{2}\Delta^2 y_2 + \frac{(t+1)t(t-1)}{6}\Delta^3 y_2 \\
&= 4.5462 + 0.220924 + 0.00012012 + 0.00006978 = 4.7673139 \approx \mathbf{4.7673}
\end{aligned}$$

1.5 Интерполяция многочленом Лагранжа

Для нахождения значений в точках X_1 и X_2 применим полином Лагранжа второй степени, построенный по трем ближайшим к каждой точке узлам.

Вычисление для $X_1 = 0.761$

Узлы:

- $(x_4, y_4) = (0.70, 5.5504)$
- $(x_5, y_5) = (0.75, 6.5559)$
- $(x_6, y_6) = (0.80, 7.5594)$

Расчёт базисных полиномов:

$$\begin{aligned}l_4(0.761) &= \frac{(0.761 - 0.75)(0.761 - 0.80)}{(0.70 - 0.75)(0.70 - 0.80)} = -0.0858 \\l_5(0.761) &= \frac{(0.761 - 0.70)(0.761 - 0.80)}{(0.75 - 0.70)(0.75 - 0.80)} = 0.9516 \\l_6(0.761) &= \frac{(0.761 - 0.70)(0.761 - 0.75)}{(0.80 - 0.70)(0.80 - 0.75)} = 0.1342\end{aligned}$$

Итоговое значение:

$$\begin{aligned}L_2(0.761) &= y_4 \cdot l_4 + y_5 \cdot l_5 + y_6 \cdot l_6 \\&= -0.4762 + 6.2386 + 1.0145 = \mathbf{6.7769}\end{aligned}$$

Вычисление для $X_2 = 0.661$

Узлы:

- $(x_2, y_2) = (0.60, 3.5406)$
- $(x_3, y_3) = (0.65, 4.5462)$
- $(x_4, y_4) = (0.70, 5.5504)$

Расчет базисных полиномов:

$$l_2(0.661) = \frac{(0.661 - 0.65)(0.661 - 0.70)}{(0.60 - 0.65)(0.60 - 0.70)} = -0.0858$$

$$l_3(0.661) = \frac{(0.661 - 0.60)(0.661 - 0.70)}{(0.65 - 0.60)(0.65 - 0.70)} = 0.9516$$

$$l_4(0.661) = \frac{(0.661 - 0.60)(0.661 - 0.65)}{(0.70 - 0.60)(0.70 - 0.65)} = 0.1342$$

Итоговое значение:

$$\begin{aligned} L_2(0.661) &= y_2 \cdot l_2 + y_3 \cdot l_3 + y_4 \cdot l_4 \\ &= -0.3038 + 4.3262 + 0.7449 = \mathbf{4.7673} \end{aligned}$$

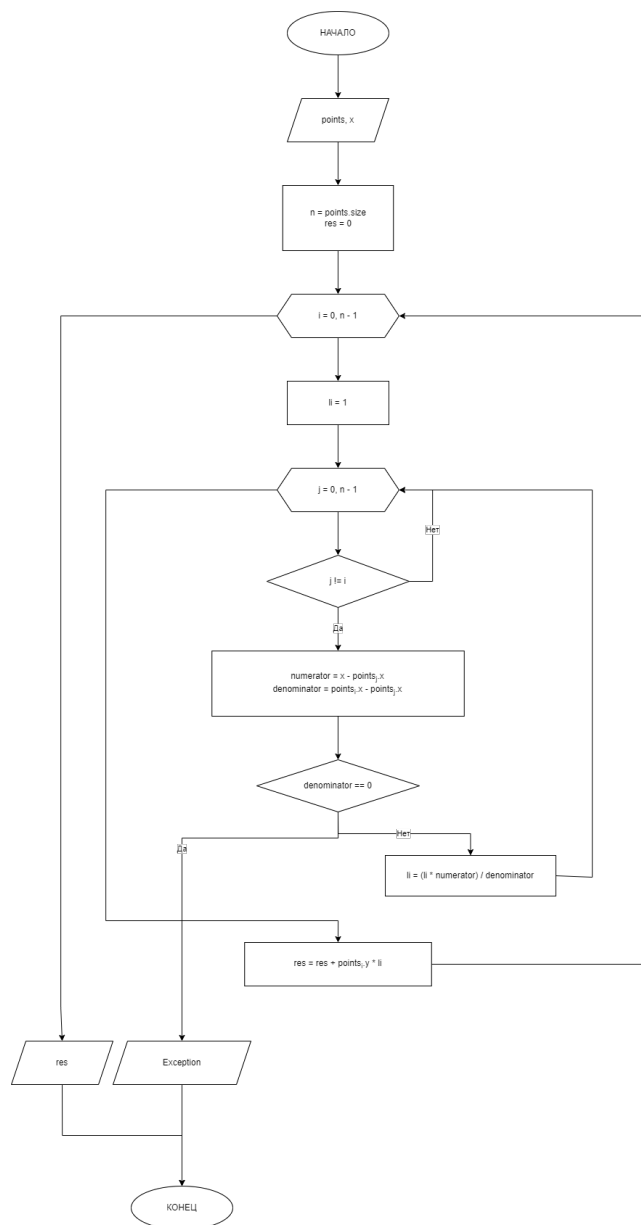
1.6 Сравнение результатов

Метод	Значение в $X_1 = 0.761$	Значение в $X_2 = 0.661$
Ньютон	6.7770	—
Гаусс	—	4.7673
Лагранж	6.7769	4.7673

Результаты, полученные разными методами, практически совпадают, что свидетельствует о корректности вычислений.

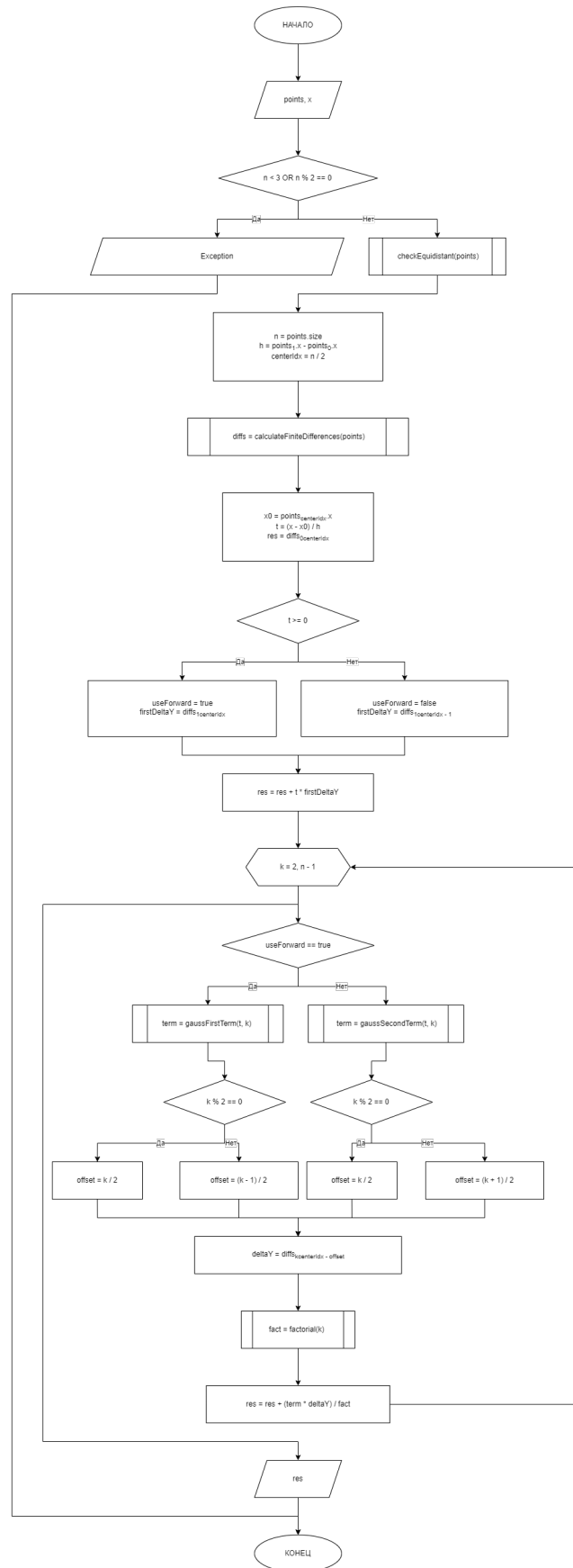
Блок-схемы алгоритмов

Лагранж

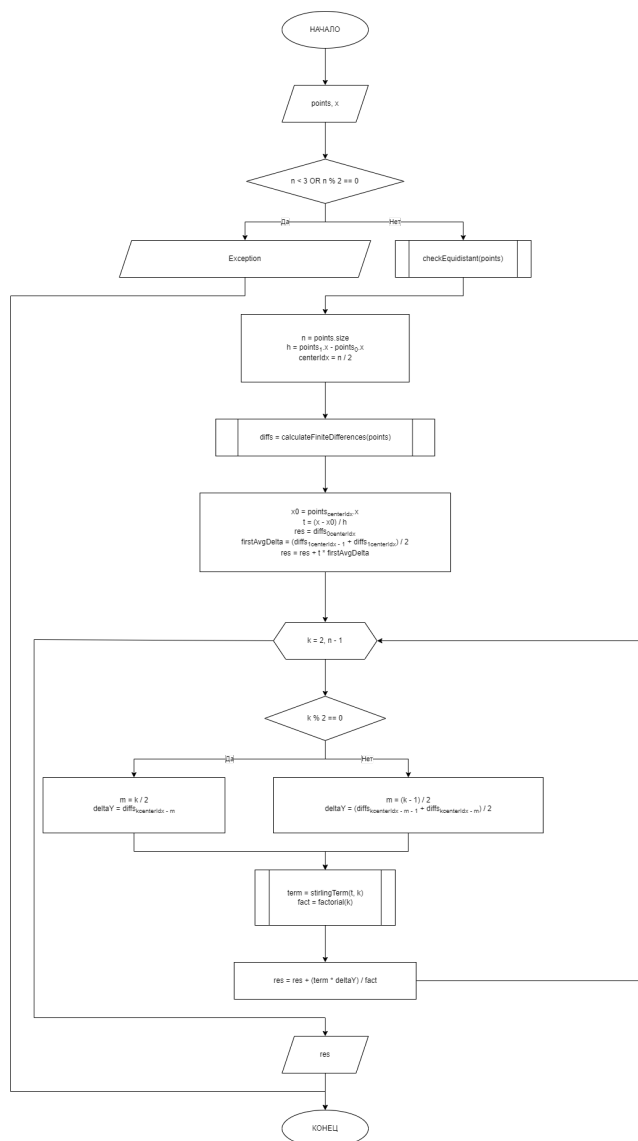


НЬЮТОН

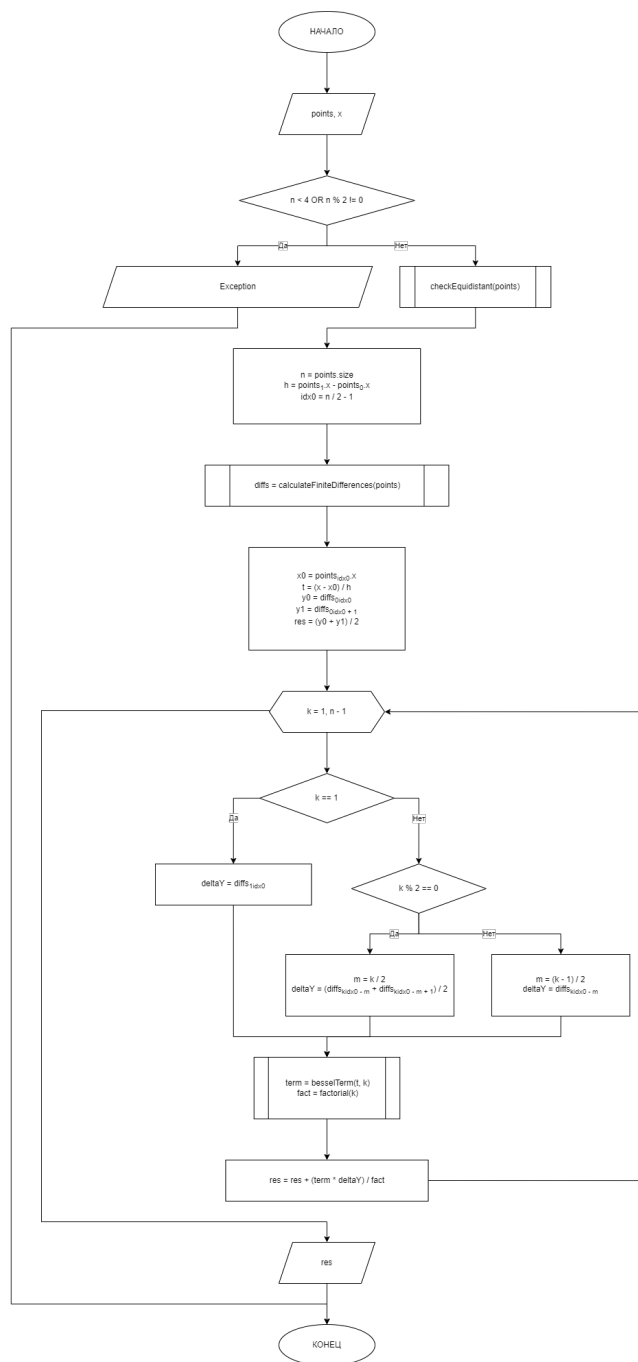


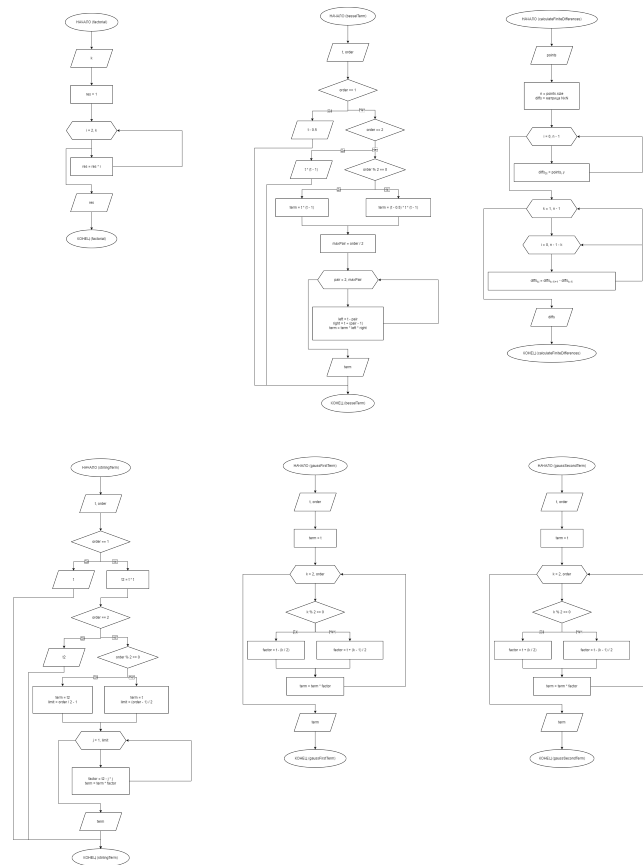


Стирлинг



Бессель





Листинг программы

Исходный код программы доступен в репозитории:

[ссылка на GitHub](#)

```

1 package arhr.tech.comp_math_lab2.services.solvers
2
3 import arhr.tech.comp_math_lab2.dto.InterpolationRequestDto
4 import arhr.tech.comp_math_lab2.dto.InterpolationResult
5 import arhr.tech.comp_math_lab2.dto.PointDto
6 import org.springframework.stereotype.Component
7 import java.math.BigDecimal
8 import java.math.MathContext
9 import java.math.RoundingMode
10
11 @Component
12 class LagranzhSolver : InterpolationSolver {
13
14     private val mc = MathContext(16, RoundingMode.HALF_UP)
15
16     override fun solve(request: InterpolationRequestDto):
17     InterpolationResult {
18         val points = request.points
19     }
20 }

```

```

18     val n = points.size
19
20     val evaluate: (BigDecimal) -> BigDecimal = { calcX ->
21         var res = BigDecimal.ZERO
22         for (i in 0 until n) {
23             var li = BigDecimal.ONE
24             for (j in 0 until n) {
25                 if (j != i) {
26                     val numerator = calcX - points[j].x
27                     val denominator = points[i].x - points[j].x
28                     require(denominator != BigDecimal.ZERO) { "
29                         X
30                         " }
31                     li = (li * numerator).divide(denominator, mc)
32                 }
33             }
34             res = res + (points[i].y * li)
35         }
36
37         val result = evaluate(request.x)
38         val graphicPoints = InterpolationSolver.generateGraphicPoints(points
39             , evaluate)
40
41         return InterpolationResult(
42             type = InterpolationType.LAGRANZH,
43             x = request.x.round(),
44             y = result.round(),
45             message = null,
46             graphicPoints = graphicPoints
47         )
48     }
49
50     override fun supports(type: InterpolationType): Boolean {
51         return type == InterpolationType.LAGRANZH
52     }
53 }
54
55
56 package arhr.tech.comp_math_lab2.services.solvers
57
58 import arhr.tech.comp_math_lab2.dto.InterpolationRequestDto
59 import arhr.tech.comp_math_lab2.dto.InterpolationResult
60 import arhr.tech.comp_math_lab2.dto.PointDto
61 import org.springframework.stereotype.Component
62 import java.math.BigDecimal
63 import java.math.MathContext

```

```

64 import java.math.RoundingMode
65
66 @Component
67 class NewtonSolver : InterpolationSolver {
68
69     private val mc = MathContext(16, RoundingMode.HALF_UP)
70
71     override fun solve(request: InterpolationRequestDto):
InterpolationResult {
72         val points = request.points
73         val x = request.x
74         val n = points.size
75
76         checkEquidistant(points)
77         val h = points[1].x - points[0].x
78
79         val diffs = InterpolationSolver.calculateFiniteDifferences(points)
80
81         val evaluate: (BigDecimal) -> BigDecimal = { calcX ->
82             val useFwd = (calcX - points[0].x).abs() <= (calcX - points.last
().x).abs()
83             val tCalc = if (useFwd) {
84                 (calcX - points[0].x).divide(h, mc)
85             } else {
86                 (calcX - points.last().x).divide(h, mc)
87             }
88
89             var res = if (useFwd) points[0].y else points.last().y
90             var termCalc = BigDecimal.ONE
91
92             for (k in 1 until n) {
93                 val delta = if (useFwd) tCalc - BigDecimal(k - 1) else tCalc
+ BigDecimal(k - 1)
94                 termCalc = termCalc.multiply(delta, mc)
95                 val deltaY = if (useFwd) diffs[k][0] else diffs[k][n - 1 - k
]
96                 val fact = factorial(k, mc)
97                 res = res.add(termCalc.multiply(deltaY, mc).divide(fact, mc)
)
98             }
99             res
100         }
101
102         val useForward = (x - points[0].x).abs() <= (x - points.last().x).
abs()
103         val result = evaluate(x)
104         val graphicPoints = InterpolationSolver.generateGraphicPoints(points
, evaluate)

```

```

105
106     return InterpolationResult(
107         type = InterpolationType.NEWTON,
108         x = x,
109         y = result,
110         message = "
111
112                                     " + if (useForward) "
113                                     " else
114         ",
115         graphicPoints = graphicPoints
116     )
117 }

118
119 override fun supports(type: InterpolationType): Boolean = type ==
InterpolationType.NEWTON

```

Вывод

В ходе выполнения данной лабораторной работы я изучил и применил на практике основные методы полиномиальной интерполяции: многочлены Лагранжа, Ньютона и Гаусса. Были произведены расчеты значений функции в заданных точках. Результаты, полученные с помощью различных методов, оказались очень близкими, что подтверждает их согласованность и правильность выполненных вычислений.